CO1 ASSIGNMENT
EXPERIMENT – 1
ANALYSIS, DEVELOPMENT AND VERIFICATION OF 8-BIT AND 16-BIT ARITHMETIC OPERATIONS USING ASSEMBLY LANGUAGE PROGRAM

Course / Subject	**Microprocessors and Microcontrollers (MPMC)**
Course & Course code	CO1 & ECA1009
Bloom's Taxonomy Level	K4 – Analyze
Work Knowledge Level	WK3 – Engineering Fundamentals
Simulator	8085 Simulator Software Kit
Student Name	B Akshaya
Register Number	192424336
Department	Artificial Intelligence and Data Science (AIDS)

This assignment includes four sub-experiments: 8-bit addition and subtraction, 16-bit addition and subtraction, 8-bit multiplication and division, and 16-bit multiplication and division.

INDEX

S. No.	Contents	Page
1	Aim, objectives, outcomes and requirements	2
2	Theory and instruction set	3
3	Sub-experiment 1: 8-bit addition and subtraction	4–5
4	Sub-experiment 2: 16-bit addition and subtraction	6–7
5	Sub-experiment 3: 8-bit multiplication and division	8–9
6	Sub-experiment 4: 16-bit multiplication and division	10–11
7	Result, precautions and viva-voce questions	12–13

1. AIM

To develop, execute and verify Intel 8085 assembly-language programs for 8-bit and 16-bit addition, subtraction, multiplication and division using operands stored in memory. The experiment also aims to store the results in specified memory locations and analyze carry, borrow and condition flags by comparing simulator output with manual hexadecimal calculations.

Objectives

- Develop assembly programs for 8-bit and 16-bit addition and subtraction.
- Implement multiplication using repeated addition and division using repeated subtraction because the 8085 has no dedicated multiplication or division instruction.
- Observe register contents, memory values and the flag register after execution.
- Distinguish unsigned carry/borrow from signed overflow.
- Verify all results through manual hexadecimal calculations.

Expected outcomes

- Obtain sum, difference, product, quotient and remainder in the specified memory locations.
- Understand carry propagation and borrow handling in multi-byte arithmetic.
- Explain how counters and conditional jumps control arithmetic loops.
- Verify division using dividend = quotient × divisor + remainder.
- Compare actual simulator observations with theoretical results.

Software / hardware required

- Computer or laptop with an Intel 8085 simulator and keyboard.
- 8085 trainer kit, if available.
- Assembly-language editor, assembler and simulator execution controls.

2. FUNDAMENTAL THEORY

The Intel 8085 is an 8-bit microprocessor. Its accumulator and general-purpose registers are 8 bits wide, while register pairs BC, DE and HL can hold 16-bit values. Addition and subtraction are performed using instructions such as ADD, ADC, SUB and SBB. The DAD instruction performs 16-bit addition by adding a register pair to HL.

The 8085 instruction set does not contain dedicated multiplication or division instructions. Multiplication is therefore implemented by repeatedly adding the multiplicand according to the multiplier. Division is implemented by repeatedly subtracting the divisor from the dividend. The number of successful subtractions gives the quotient, while the remaining value is the remainder.

Important instructions

Instruction	Purpose
LXI	Loads a 16-bit immediate value into a register pair.
LDA / STA	Loads from or stores to a direct memory address.
LHLD / SHLD	Loads or stores a 16-bit value in memory; low byte first.
MOV r,M	Copies the memory byte addressed by HL into a register.
ADD / ADC	Adds an operand to A; ADC includes carry.
SUB / SBB	Subtracts an operand from A; SBB includes borrow.
DAD rp	Adds a register pair to HL and updates carry.
CMP	Compares A with an operand by updating flags.
JZ / JNZ / JC / JNC	Conditional jumps based on flags.
PUSH PSW / POP	Saves/restores accumulator and flag information via stack.
HLT	Stops program execution.

Flag register

The 8085 has Sign (S), Zero (Z), Auxiliary Carry (AC), Parity (P) and Carry (CY) flags. S reflects the most significant bit of a result. Z is set when the result is zero. P is set when the result contains an even number of 1-bits. AC relates to carry between the lower and upper nibbles. CY indicates unsigned carry in addition or borrow in subtraction. The 8085 has no separate signed-overflow flag; signed overflow must be analyzed from the operands and result.

Memory convention: 16-bit data are stored in little-endian order. For example, 1234H is stored as 34H at the lower address and 12H at the next address.

3. SUB-EXPERIMENT 1 — 8-BIT ADDITION AND SUBTRACTION

Aim

To add and subtract two 8-bit hexadecimal operands, store the result and carry/borrow indicators, and observe the flag register.

Input and output memory map

Address	Description	Value
4500H	Operand A	9CH
4501H	Operand B	7AH
4502H	Sum	16H
4503H	Addition carry	01H
4504H	Flags after ADD	Record actual simulator value
4505H	Difference A − B	22H
4506H	Subtraction borrow	00H
4507H	Flags after SUB	Record actual simulator value

Theory and calculation

Addition: 9CH + 7AH = 116H. The lower eight bits give 16H, and the carry is 01H. Subtraction: 9CH − 7AH = 22H. As an unsigned operation, A is greater than B, so the final borrow is zero.

Algorithm

- Initialize the stack pointer and point HL to 4500H.
- Load operand A into the accumulator and operand B into register B.
- Execute ADD B, save the accumulator and flags, then store the sum and carry.
- Reload A, execute SUB B, save the accumulator and flags, then store the difference and borrow.
- Stop execution and inspect addresses 4502H–4507H.

Assembly-language program (starting address 4100H)

```
LXI SP,47FFH
LXI H,4500H
MOV A,M
INX H
MOV B,M
```

```
ADD B
PUSH PSW
STA 4502H
MVI A,00H
ACI 00H
STA 4503H
POP H
MOV A,L
STA 4504H
LDA 4500H
SUB B
PUSH PSW
STA 4505H
MVI A,00H
ACI 00H
STA 4506H
POP H
MOV A,L
STA 4507H
HLT
```

Procedure

- Open the 8085 simulator and enter the code.
- Set starting address to 4100H and assemble the program.
- Enter 9CH at 4500H and 7AH at 4501H.
- Execute the program using GO/EXEC.
- Inspect 4502H–4507H and record the actual values.
- Compare observed results with theoretical calculations.

Expected output

Operation	Expected result
Addition	Sum = 16H; carry = 01H
Subtraction	Difference = 22H; borrow = 00H

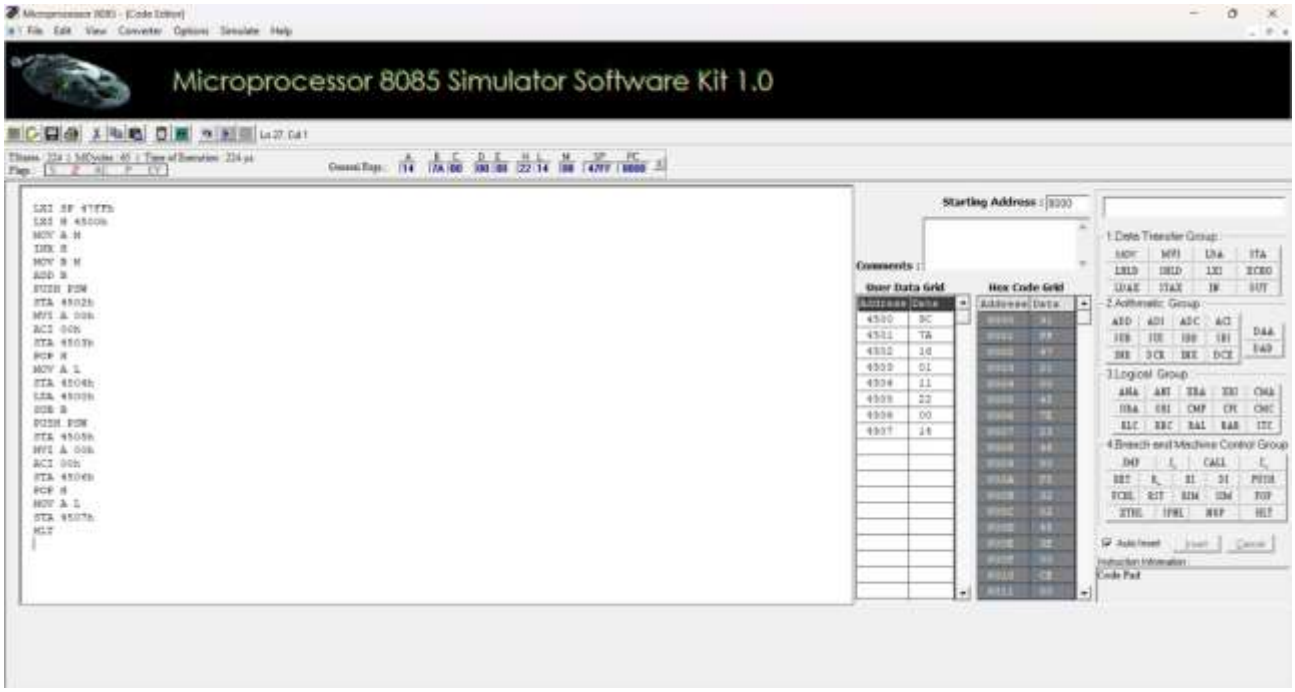


Figure 1. 8085 simulator screenshot for Sub-experiment 1: 8-bit addition and subtraction.

4. SUB-EXPERIMENT 2 — 16-BIT ADDITION AND SUBTRACTION

Aim

To perform 16-bit addition and subtraction using register pairs and byte-wise arithmetic, then store the results and carry/borrow indicator in memory.

Input and output memory map

Address	Description	Value
4510H–4511H	X = 1234H (low byte first)	34H, 12H
4512H–4513H	Y = 5678H (low byte first)	78H, 56H
4514H–4515H	Sum X + Y	68ACH
4516H	Addition carry	00H
4518H–4519H	Difference X − Y	BBBCH
451AH	Subtraction borrow	01H

Theory and calculation

Addition: 1234H + 5678H = 68ACH. The result is below 10000H, so the carry is zero. Subtraction: 1234H − 5678H = BBBCH in 16-bit two's-complement representation. Since X is less than Y as an unsigned number, the subtraction generates a borrow.

Algorithm

- Load X into HL and exchange it into DE.
- Load Y into HL and clear register C.
- Use DAD D to add X and Y; store the result and carry.
- Subtract the lower bytes using SUB and the upper bytes using SBB to include the borrow.
- Store the difference and borrow indicator, then halt.

Assembly-language program (starting address 4200H)

```
LHLD 4510H
XCHG
LHLD 4512H
MVI C,00H
DAD D
JNC SKIP
INR C
```

```
SKIP: SHLD 4514H
MOV A,C
STA 4516H
MVI C,00H
LDA 4510H
LXI H,4512H
SUB M
STA 4518H
LDA 4511H
INX H
SBB M
STA 4519H
JNC STORE
INR C
STORE: MOV A,C
STA 451AH
HLT
```

Procedure

- Set the starting address to 4200H and enter Program 2.
- Enter X = 1234H at 4510H–4511H in low-byte-first order.
- Enter Y = 5678H at 4512H–4513H in low-byte-first order.
- Assemble and execute the program.
- Inspect 4514H–4516H and 4518H–451AH.
- Verify the combined 16-bit results manually.

Expected output

Operation	Expected result
Addition	Sum = 68ACH; carry = 00H
Subtraction	Difference = BBBCH; borrow = 01H

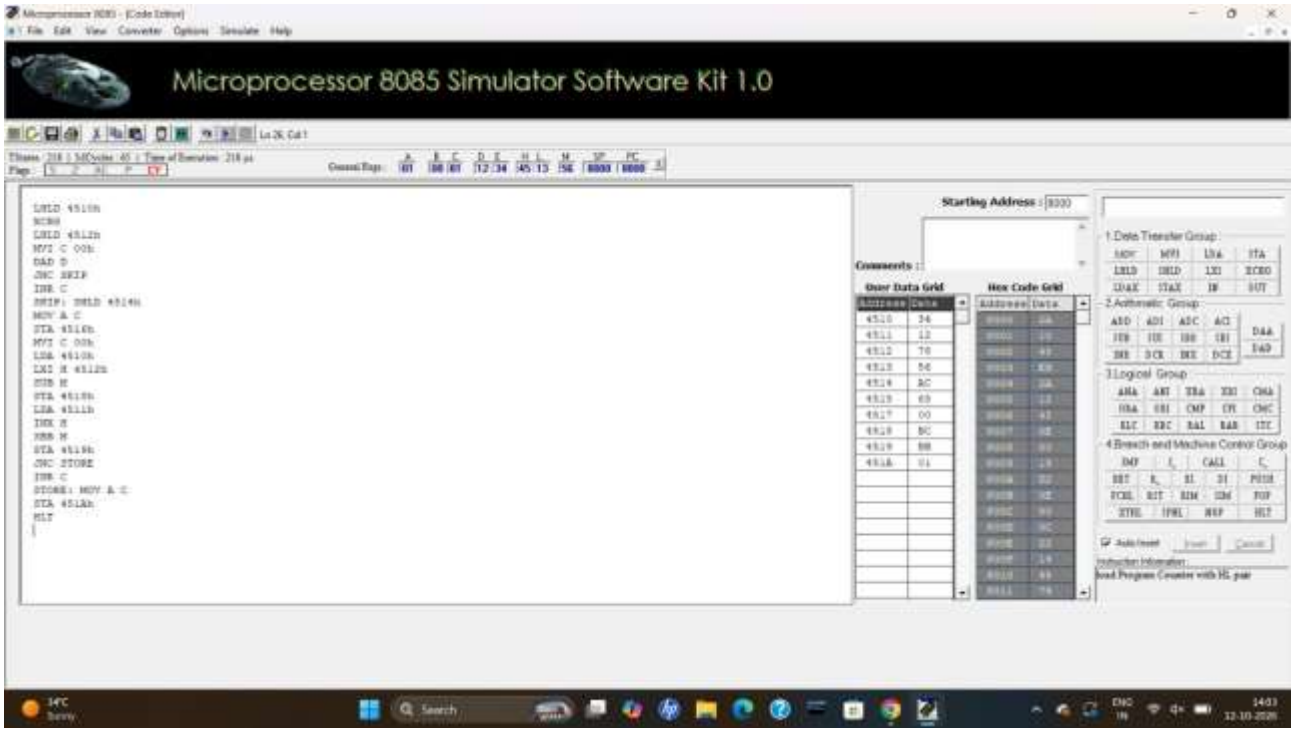


Figure 2. 8085 simulator screenshot for Sub-experiment 2: 16-bit addition and subtraction.

5. SUB-EXPERIMENT 3 — 8-BIT MULTIPLICATION AND DIVISION

Aim

To implement 8-bit multiplication by repeated addition and 8-bit division by repeated subtraction, storing the product, quotient and remainder.

Input and output memory map

Address	Description	Value
4520H	Multiplicand	12H
4521H	Multiplier	05H
4522H–4523H	16-bit product	005AH
4524H	Dividend	64H
4525H	Divisor	05H
4526H	Quotient	14H
4527H	Remainder	00H

Theory and calculation

Multiplication: 12H × 05H = 5AH. The program adds 12H to the product five times. Division: 64H ÷ 05H = 14H with remainder 00H. Repeated subtraction subtracts 05H from 64H twenty times. Verification: 64H = 14H × 05H + 00H.

Algorithm

- Load the multiplicand into E, clear D and load the multiplier into counter C.
- Initialize HL to zero and repeatedly add DE until the counter reaches zero.
- Store the 16-bit product at 4522H–4523H.
- Load the divisor into B, the dividend into A and clear C for the quotient.
- Compare A with B; subtract and increment C while A is at least B.
- Store the remainder and quotient, then halt.

Assembly-language program (starting address 4300H)

```
LDA 4520H
MOV E,A
MVI D,00H
LDA 4521H
MOV C,A
LXI H,0000H
ORA A
JZ STORE
LOOP: DAD D
DCR C
JNZ LOOP
STORE: SHLD 4522H
LDA 4525H
MOV B,A
LDA 4524H
MVI C,00H
NEXT: CMP B
JC DONE
SUB B
INR C
JMP NEXT
DONE: STA 4527H
MOV A,C
STA 4526H
HLT
```

Procedure

- Set the starting address to 4300H and enter the code.
- Enter 12H at 4520H, 05H at 4521H, 64H at 4524H and 05H at 4525H.
- Assemble and execute the program.

• Read the product at 4522H–4523H, quotient at 4526H and remainder at 4527H.

• Verify dividend = quotient × divisor + remainder and remainder < divisor.

Expected output

Operation	Expected result
Multiplication	Product = 005AH
Division	Quotient = 14H; remainder = 00H

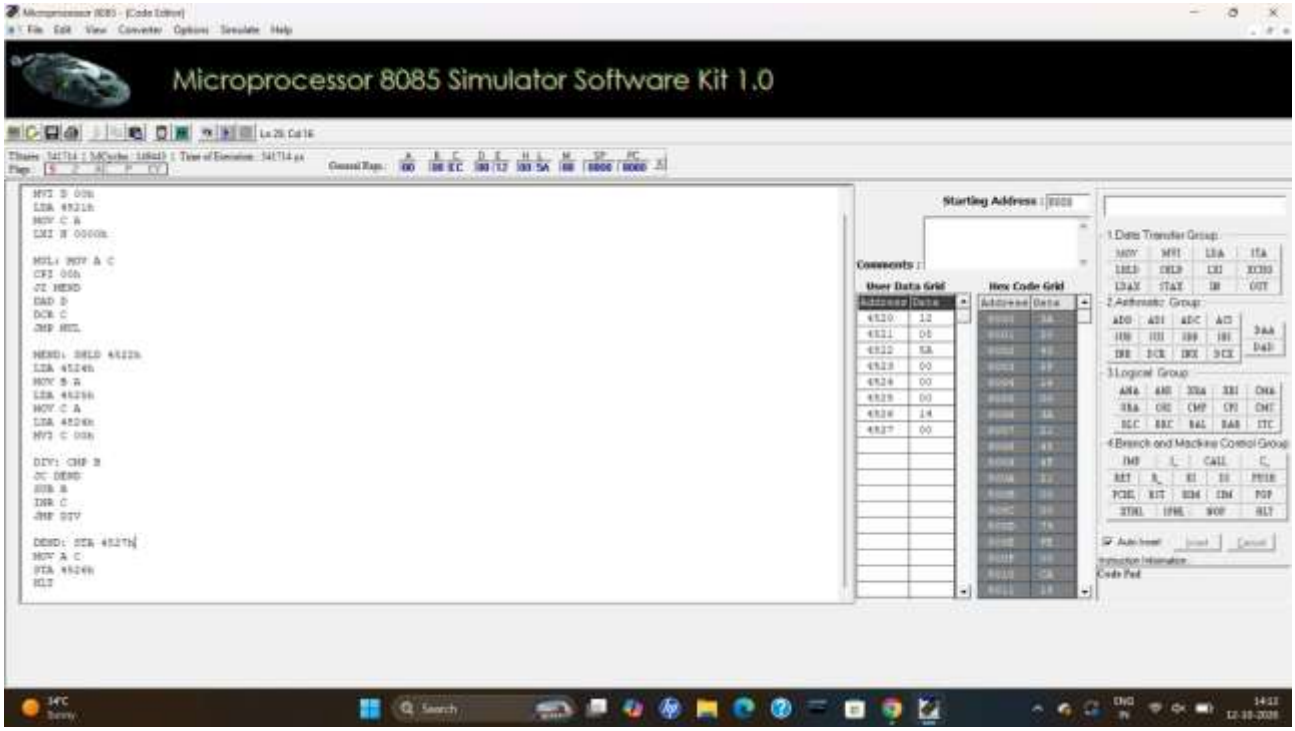


Figure 3. 8085 simulator screenshot for Sub-experiment 3: multiplication and division.

6. SUB-EXPERIMENT 4 — 16-BIT MULTIPLICATION AND DIVISION

Aim

To implement 16-bit multiplication through repeated addition with carry propagation and 16-bit division through repeated subtraction, storing the product, quotient and remainder.

Input and output memory map

Address	Description	Value
4530H–4531H	Multiplicand X = 1234H	34H, 12H
4532H–4533H	Multiplier Y = 5678H	78H, 56H
4534H–4537H	32-bit product	06260060H
4538H–4539H	Dividend = 5678H	78H, 56H
453AH–453BH	Divisor = 1234H	34H, 12H
453CH–453DH	Quotient	0004H
453EH–453FH	Remainder	0DA8H

Theory and calculation

Multiplication: 1234H × 5678H = 06260060H. The lower 16-bit word is 0060H and the upper word is 0626H. Division: 5678H ÷ 1234H = quotient 0004H and remainder 0DA8H. Verification: 4 × 1234H + 0DA8H = 5678H, and the remainder is less than the divisor.

Algorithm

- Load the multiplicand into DE and the multiplier into BC as the loop counter.
- Clear HL and initialize the upper product word to zero.
- Add DE repeatedly to HL; increment the upper word whenever DAD generates carry.
- Store the 32-bit product in memory.
- Load divisor into DE and dividend into HL; clear BC for the quotient.
- Subtract the divisor using SUB and SBB; if borrow occurs, restore the remainder by adding the divisor back.
- Store quotient and remainder, then halt.

Assembly-language program (starting address 4400H)

```
LXI SP,47FFH
LHLD 4530H
XCHG
LHLD 4532H
MOV B,H
MOV C,L
LXI H,0000H
SHLD 4536H
MOV A,B
ORA C
JZ STORE
MLOOP: DAD D
JNC NOCY
PUSH H
LHLD 4536H
INX H
SHLD 4536H
POP H
NOCY: DCX B
MOV A,B
ORA C
JNZ MLOOP
STORE: SHLD 4534H
LHLD 453AH
XCHG
LHLD 4538H
LXI B,0000H
DLOOP: MOV A,L
SUB E
MOV L,A
MOV A,H
SBB D
```

```
MOV H,A
JC DONE
INX B
JMP DLOOP
DONE: DAD D
SHLD 453EH
MOV H,B
MOV L,C
SHLD 453CH
HLT
```

Procedure

- Set the starting address to 4400H and enter the code.
- Enter X = 1234H at 4530H–4531H and Y = 5678H at 4532H–4533H, low byte first.
- Enter dividend 5678H at 4538H–4539H and divisor 1234H at 453AH–453BH.
- Assemble and execute; the multiplication loop may take longer.
- Inspect 4534H–4537H for the product, 453CH–453DH for quotient and 453EH–453FH for remainder.
- Verify dividend = quotient × divisor + remainder.

Expected output

Operation	Expected result
Multiplication	32-bit product = 06260060H
Division	Quotient = 0004H; remainder = 0DA8H

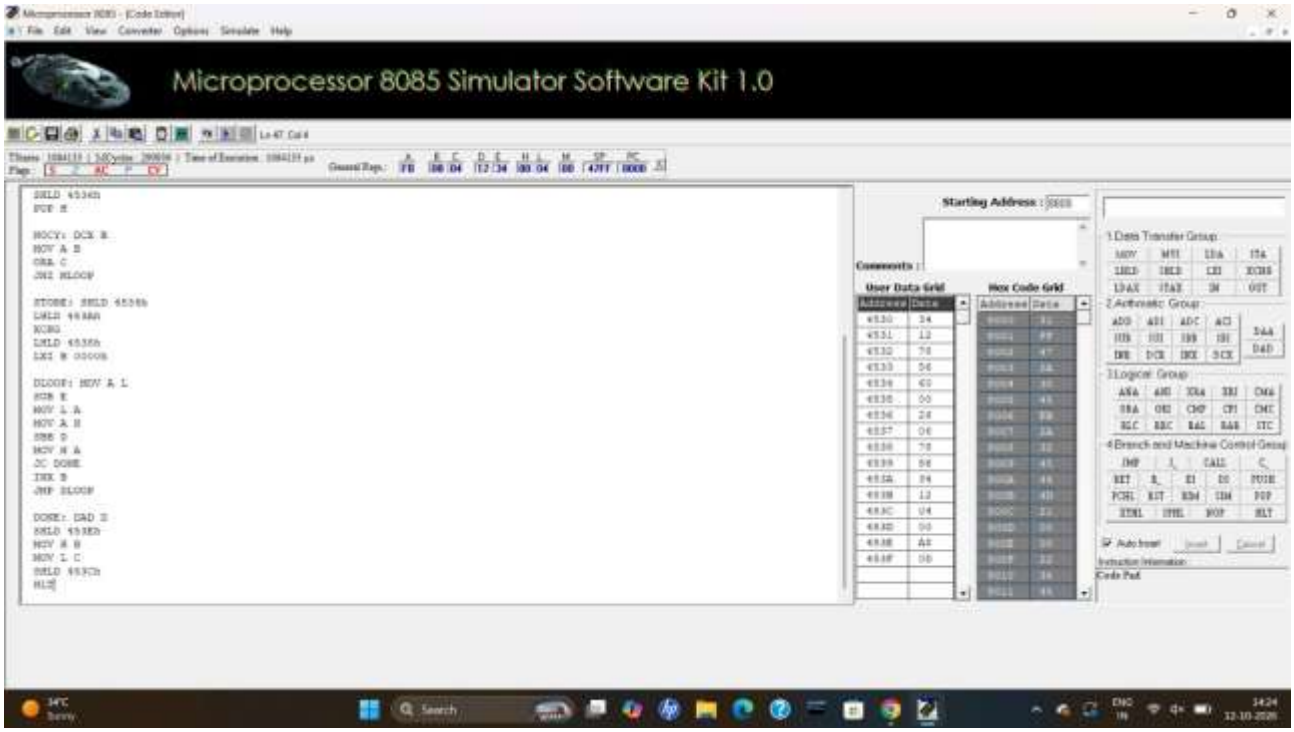


Figure 4. 8085 simulator screenshot for Sub-experiment 4: 16-bit multiplication and division.

7. COMBINED OBSERVATION TABLE

S. No.	Operation	Input	Expected output
1	8-bit addition	9CH + 7AH	16H; carry 01H
2	8-bit subtraction	9CH − 7AH	22H; borrow 00H
3	16-bit addition	1234H + 5678H	68ACH; carry 00H
4	16-bit subtraction	1234H − 5678H	BBBCH; borrow 01H
5	8-bit multiplication	12H × 05H	005AH
6	8-bit division	64H ÷ 05H	Q=14H; R=00H
7	16-bit multiplication	1234H × 5678H	06260060H
8	16-bit division	5678H ÷ 1234H	Q=0004H; R=0DA8H

8. RESULT

The four sub-experiments cover 8-bit and 16-bit addition, subtraction, multiplication and division using Intel 8085 assembly-language programming. The programs demonstrate arithmetic operations, register-pair usage, carry/borrow handling, memory storage and loop-based implementation of multiplication and division. The experiment is verified when the actual simulator outputs match the theoretical results.

- Use the correct program starting addresses: 4100H, 4200H, 4300H and 4400H.
- Enter 16-bit values in little-endian order (lower byte first).
- Initialize registers, counters and the stack pointer before use.
- Avoid overlapping code, stack, input and output memory regions.
- Do not divide by zero.
- Check conditional jumps and loop termination to avoid infinite loops.
- Record actual simulator outputs rather than copying expected values as observations.